

Aula 5 — Maestro E2E em RN + Lab Perf/Security + Trabalho Final

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 02/07/2026

Agenda de hoje (3h30)

1. **Plantão de setup + Lab perf/security** — rodam em paralelo (~40min)
2. Recap da pirâmide de testes
3. **Maestro sobre o app RN** — fechar os 5 flows (Atividade 4)
4. Intervalo
5. **Trabalho Final** — revelar temas + rubrica
6. Combinar a data da apresentação

Como vamos usar os próximos ~40min

Ainda ajustando o Android Studio/emulador? Chama o prof — resolvemos juntos, com calma.

Device/emulador já de pé? Começa o **lab de perf/security**, sozinho:

`exercicios/03-maestro-e2e/lab-perf-security.md`

Sem device ainda? Começa pela parte do lab que **não precisa de adb** — achar e corrigir o manifest.

| Todo mundo com algo pra fazer em paralelo. Reconvergimos em ~40min.

O app-alvo: CineFav

`com.puciec.cinefav` — abre na tela de **Login**.

Login mock: **qualquer email com @ + senha 4+ caracteres**.

Ex.: `aluno@puc.br` / `1234`.

Dados **mockados** por padrão (sem token TMDB, sem rede) — flows **determinísticos**.

Matrix sempre existe.

Lab — Performance + Security no CineFav

Self-guided. Sem slide teórico genérico — medindo o app que vocês já têm rodando.

0. Antes de começar

`adb shell ...` roda **de qualquer pasta** — só precisa do device conectado. Só o que mexe em arquivo do repo (manifest, `./gradlew`) exige `cd exercicios/03-maestro-e2e/pratica`.

```
adb devices    # esperado: "emulator-5554 device" (ou serial do celular)
```

Travou? `adb devices` vazio → reabre emulador/religa cabo. `adb: command not found` → falta `ANDROID_HOME` no PATH (ver `COMECE-AQUI.md`). Comando trava → `adb kill-server && adb start-server`.

Performance — cold start (quem tem device/emulador)

```
adb shell am force-stop com.puciec.cinefav  
adb shell am start -W com.puciec.cinefav/.MainActivity
```

Olhar **TotalTime** (ms). Rodar 3x — testamos agora: **1935 → 2612 → 3521ms**, variação normal em emulador.

⚠ **Warm start não é só "tirar o force-stop"** — se o app ainda tá aberto, **TotalTime** vem 0 (não mede nada). Manda pra background primeiro: `adb shell input keyevent KEYCODE_HOME`, depois roda o `am start` de novo. Testamos: **280ms** (vs ~2000-3500ms cold — 6x+ mais rápido).

Performance — jank ao rolar (quem tem device/emulador)

Rolar a lista de filmes no device, depois:

```
adb shell dumpsys gfxinfo com.puciec.cinefav
```

Olhar **Janky frames** (% acima de 16ms/60fps). Testamos agora: **98.51% em emulador** — número alto é esperado sem GPU real, não é o app. Comparem com quem tá em device físico (bem menor).

Security — achado real no manifest (todo mundo, sem adb)

```
cd exercicios/03-maestro-e2e/pratica
cat android/app/src/main/AndroidManifest.xml \
  | grep -E "allowBackup|EXTERNAL_STORAGE|SYSTEM_ALERT"
```

Dois achados reais (não plantados):

1. `allowBackup="true"` — extrai dados do app via `adb backup` sem root (OWASP M9/M2, MASVS-STORAGE)
2. `READ/WRITE_EXTERNAL_STORAGE` + `SYSTEM_ALERT_WINDOW` — permissões que o app não usa

Security — fix (achar+editar todo mundo — rebuild só quem tem device)

```
- android:allowBackup="true"  
+ android:allowBackup="false"
```

- remover as 3 permissões não usadas.

Sem device? Prova o achado no `CineFav.apk` já baixado, sem buildar nada:

```
aapt dump xmltree CineFav.apk AndroidManifest.xml | grep -i backup — mostra 0xffffffff  
(= true), não o texto "true". Depois do fix vira 0x00000000 (= false).
```

```
npm install    # se instalou via APK pronto, nunca rodou isso aqui — build falha sem  
cd android && ./gradlew assembleDebug    # ~5min, pesado em disco  
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

Achar → corrigir → reverificar. É literalmente o que MASVS/OWASP Mobile Top 10 pedem — vocês acabaram de fazer isso de verdade, sozinhos.

Recap — pirâmide de testes no CineFav

Camada	O que testamos	Como	Status
Unit	funções puras, stores Zustand	Jest, sem simulador	✓
Integração	fluxo entre componentes (RNTL, DOM emulado)	Jest + RNTL, mock	✓
E2E	app de verdade, abrindo no device	Maestro	agora

| Agora é a única camada que abre o app **real** — sem emular nada.

testIDs — a ponte entre RNTL e Maestro

No componente RN:

```
<Pressable testID={`movie-card-${movie.id}`} onPress={...}>
```

Vocês já usaram esse **mesmo testID** na Atividade 2/3 (RNTL):

```
screen.getByTestId('movie-card-603')
```

No Maestro:

```
- tapOn:  
  id: "movie-card-603"
```

Mesmo testID, três camadas. Isso não é coincidência — é a instrumentação de acessibilidade/teste do componente RN sendo reaproveitada.

tapOn: id: vs tapOn: "texto"

```
# Frágil – quebra se o texto mudar (il8n, dado dinâmico)
- tapOn: "Matrix"

# Estável – aponta pro elemento, não pro conteúdo
- tapOn:
  id: "movie-card-603"
```

Mesmo argumento de `getByTestId` vs `getByText` no RNTL. Prefira ID quando disponível.

Accessibility role — o mesmo elemento, duas lentes

```
- tapOn:  
  id: "movie-card-heart-603"
```

Testa que o elemento existe. Mas o **leitor de tela** de um usuário cego não vê ID — vê:

```
accessibilityRole="button"  
accessibilityLabel="Adicionar favorito"
```

Se o componente RN não tiver esses props, o teste de ID passa mas a **acessibilidade real falha**.

Testar por ID garante que existe; testar por role/label garante que é **usável**.

Os 5 flows da Atividade 4

Flow	Testa	Conceito novo
01-launch.yaml	login + lista aparece	modelo resolvido 
02-search.yaml	buscar filme	env (SEARCH_TERM)
03-favorite.yaml	favoritar → conferir em Favoritos	assert cross-tela
04-detail.yaml	abrir detalhe, favoritar, voltar	navegação
05-js-dynamic.yaml	termo de busca via JS	evalScript

01 – 03 fazemos **juntos, ao vivo**. 04 – 05 + bônus fragment: solo, em casa (prazo 09/07).

Rodando

```
cd exercicios/03-maestro-e2e/pratica  
maestro test flows/01-launch.yaml  
maestro studio          # editor visual – localhost:9999
```

Celular físico? Pule `maestro-local.sh` (ele tenta bootar emulador) — vá direto no `maestro test`.

Maestro além do básico

O que os flows 01-05 já usam, e vocês vão entender agora

Maestro Studio — o que é

Editor visual **embutido** no CLI — abre em `localhost:9999` quando você roda `maestro studio`.

- Inspecciona a **hierarquia de elementos** da tela (igual DevTools, mas pro app mobile)
- Clica num elemento → mostra `id`, `text`, `bounds` — acha o seletor **sem adivinhar**
- Grava sua interação e gera esqueleto de YAML

```
maestro studio
```

Sem Studio, achar `id` certo é tentativa e erro no código-fonte. Com Studio, é apontar e clicar.

Maestro Studio — achando o seletor do 02-search

1. maestro studio com o app aberto na tela de busca
2. Clica no campo de busca na preview → painel mostra `id: "search-input"`
3. Clica no botão de busca → `id: "movielist-search-button"`
4. Copia os IDs pro flow — **é assim que o 02-search.yaml foi escrito originalmente**

Vocês vão fazer exatamente isso ao vivo daqui a pouco, construindo o 02-search.yaml juntos.

Maestro Studio — gravar/replay

Studio também **grava** sua sequência de toques e gera os comandos correspondentes — útil pra rascunhar rápido um flow novo.

Não é copiar-colar cego. O que sai do gravador geralmente precisa de limpeza: trocar `tap0n: "texto"` por `tap0n: { id: ... }` (mais estável), remover passo redundante. Studio acelera o rascunho, não substitui revisar o flow.

Fluxo condicional — `runFlow: when:`

```
- runFlow:  
  when:  
    visible:  
      id: "login-screen"  
      file: _fragments/login.yaml
```

Por quê: se o app já estiver logado (sessão de execução anterior, ou reexecução do mesmo flow), a tela de login não aparece — sem o `when:`, o flow tentaria tocar num elemento que não existe e quebraria. Com `when:`, só faz login **se precisar**.

Isso é o mesmo `id` do teste RNTL — `login-screen` é o mesmo testID que vocês já usam desde a Atividade 2.

Fluxo condicional — aplicado nos 5 flows

Todos os 01 – 05 chamam o login assim — é o mesmo padrão repetido de propósito:

```
- launchApp
- runFlow:
  when:
    visible:
      id: "login-screen"
  file: _fragments/login.yaml
```

Roda o flow 2x seguidas sem reinstalar o app — na 2ª vez, login é pulado automaticamente.

Testem isso ao vivo depois.

Fragments — DRY em testes E2E

`_fragments/login.yaml` — 1 arquivo, chamado pelos 5 flows via `runFlow: file::`

```
# _fragments/login.yaml
appId: com.puciec.cinefav
---
- tap0n: { id: "login-email-input" }
- inputText: "aluno@puc.br"
- tap0n: { id: "login-password-input" }
- inputText: "1234"
- tap0n: { id: "login-submit-button" }
```

Por quê: se o fluxo de login mudar (novo campo, botão reposicionado), muda em **1 lugar**, não em 5 arquivos. Mesmo princípio de "não repita conhecimento" que vocês já usam com função auxiliar no código — aqui é a mesma ideia aplicada a teste.

Gancho pro Trabalho Final: **tema 2 (Arquitetura de Suíte)** é levar essa ideia de reuso ainda mais longe — Robot Pattern organiza isso de forma ainda mais estruturada.

JS embutido — `evalScript` vs `runScript`

Dois jeitos de rodar JavaScript dentro de um flow:

- `evalScript` — JS *inline*, direto no YAML. Bom pra lógica curta.
- `runScript` — JS de *arquivo separado* (`.js`). Bom pra lógica maior/reutilizável.

```
# evalScript: inline
- evalScript: ${output.termo = "Matrix"}

# runScript: arquivo externo
- runScript: gerar-termo.js
```


JS embutido — exemplo real (05-js-dynamic.yaml)

```
- evalScript: ${output.termo = ['Matrix', 'Avatar', 'Interestelar'][Math.floor(Math.random() * 3)]}
- tap0n: { id: "movielist-search-button" }
- tap0n: { id: "search-input" }
- inputText: ${output.termo}
- assertVisible: ${output.termo}
```

Escolhe um título **aleatório** entre 3 opções e busca por ele — o mesmo flow testa termos diferentes a cada execução, sem hardcodar.

JS embutido — `${output.campo}`

`output` é o objeto onde `evalScript` / `runScript` **gravam resultado** — e `${output.campo}` lê esse valor no resto do flow, em qualquer step depois:

```
- evalScript: ${output.termo = "Matrix"}  
- inputText: ${output.termo}           # usa o valor gravado acima  
- assertVisible: ${output.termo}      # usa de novo
```

Mesmo princípio do `${SEARCH_TERM}` do `env:` (próximo slide) — interpolação de variável dentro do YAML. JS só entra quando a lógica é complexa demais pra um valor fixo.

Parametrização com `env:`

```
# 02-search.yaml
env:
  SEARCH_TERM: "Matrix"
---
- inputText: ${SEARCH_TERM}
- assertVisible: ${SEARCH_TERM}
```

Roda normal (usa o default `"Matrix"`), ou sobrescreve na hora:

```
maestro test flows/02-search.yaml -e SEARCH_TERM=Avatar
```

Por quê: mesmo flow, dado diferente — sem duplicar arquivo pra cada termo de busca. Data-driven testing, mesma ideia de parametrizar teste que vocês já usam em Jest (`it.each`).

Por que Maestro não precisa de `sleep()` manual

Diferente de Selenium/Appium clássico, `tapOn / assertVisible` do Maestro **já esperam** o elemento aparecer (timeout configurável) — não precisa de `sleep(2000)` espalhado pelo flow.

É por isso que E2E ainda assim pode ser flaky (próximo slide) — o wait automático ajuda com timing, mas não resolve tudo (rede lenta, animação, estado inesperado do device).

Recap — o que cada flow já ensinou

Flow	Recurso Maestro
01-launch.yaml	runFlow: when: (condicional) + fragment de login
02-search.yaml	env: parametrizado + Maestro Studio pra achar seletor
03-favorite.yaml	assert cross-tela (favoritar → conferir em outra tela)
04-detail.yaml	navegação (tap0n → tela nova → tap0n voltar)
05-js-dynamic.yaml	evalScript + \${output.campo}

Vocês não estavam só "copiando comando" — cada flow carrega um conceito. Agora sabem qual.

E2E é caro e às vezes flaky

Quando falha, o Maestro mostra exatamente onde:

```
x Unable to find element with text: "Spider-Man"
```

Rodar de novo — às vezes passa (flake real, aconteceu na última aula). **Retry existe por isso.** Diferente de unit/integração (determinístico), E2E depende do device/emulador de verdade.

Trabalho Final

60 pts · grupo de 3–4 · 1 dos 12 temas (5 trilhas)

12 temas, 5 trilhas — escolha pela skill, não só pela IA

Revisado hoje: temas contra evidência real de mercado. **Tema 2 mudou** (Espresso/XCUITest nunca foi praticado em aula) e **2 temas novos entraram** — cobrindo perfis além de "automation engineer puro".

Trilha Automação & Arquitetura

#	Tema	Stack
1	Automação Mobile E2E	Maestro + Cloud Devices
2	Arquitetura de Suíte de Teste <i>(era Native UI)</i>	Robot ou Screenplay Pattern sobre Maestro+RNTL
9	Mobile API Contract Testing	Pact + Jest

 Trilha IA aplicada a testes

#	Tema	Stack
5	Visual AI em Mobile	Appliteools/Percy
6	Test Generation com LLM	Claude API + Maestro
7	AI Agent Exploratory	AppAgent/DroidBot + LLM

 Trilha Performance & Segurança

#	Tema	Stack
3	Performance Mobile Testing — aprofunda o lab de hoje	Macrobenchmark
4	Mobile Security Testing — aprofunda o lab de hoje	OWASP MASVS + MobSF

 Trilha Qualidade & Pipeline

#	Tema	Stack
8	CI/CD Pipeline Mobile	Actions+Fastlane+Firebase (nível avançado: flaky quarantine + DORA CFR)
11	Mutation Testing (novo)	Stryker sobre lib de domínio, gate de CI

Trilha Manual, Exploratório & Acessibilidade

#	Tema	Stack
10	Accessibility Testing Mobile	a11y Scanner + Inspector + WCAG manual
12	Manual → Regressão Automatizada (novo)	Charter/SBTM (Aula 2) + conversão em teste automatizado no CI

Tema 12 é pra quem curte testar manualmente e não quer o projeto inteiro em código —
Parte A (manual) + Parte B (automação, avaliável em CI).

Onde ler tudo

Enunciado completo (com a coluna "por que é real" — fontes/evidência por tema):
`exercicios/04-trabalho-final/enunciado.md` (público, no repo).

Rubrica (60 pts)

Critério	Pts
Apresentação oral (10min + 5min Q&A)	12
Artefato técnico (repo funcional)	21
Relatório analítico (1pg)	12
Domínio conceitual em Q&A	9
Originalidade e profundidade	6

Entrega: 15/07/2026 (já no Canvas). **Data da apresentação: combinamos agora.**

Combinando a data da apresentação

Aula final = apresentações + IA em testes/CI-CD (conteúdo remanescente).

Falta só isso.